▶ 작업 보고서: 게시글 페이징 단계별 학습 - Step 1

- **작업 일시**: 2026-03-20
- **진행 단계**: 완료 (Step 1: 데이터 준비 및 기본 목록 보기)

1. 🌊 전체 작업 흐름 (Workflow)

```
1. [DB] data.sql에 테스트 데이터 20개 준비
2. [DTO] 목록 보기를 위한 ListDTO 설계 (BoardResponse)
3. [Service] '게시글목록보기()' 메서드 구현 (한글 메서드명)
4. [Controller] '/board/list' 매핑 및 모델 데이터 전달
5. [View] Mustache를 활용한 전체 목록 출력 화면(list.mustache) 구현
```

UI Mockup

```
+-------------------------------------------------------+
| [Blog] (Navbar)  글쓰기  회원정보수정  로그아웃        |
+-------------------------------------------------------+
|                                                       |
|      +-----------------------------------------+      |
|      |              게시글 목록                |      |
|      |-----------------------------------------|      |
|      |  번호  |  제목                |  상세보기 |      |
|      |-----------------------------------------|      |
|      |  20    |  스무 번째 게시글    | [상세보기]|      |
|      |  19    |  열아홉 번째 게시글  | [상세보기]|      |
|      |  ...   |  ...                 | [상세보기]|      |
|      |  1     |  첫 번째 게시글      | [상세보기]|      |
|      +-----------------------------------------+      |
|                                                       |
+-------------------------------------------------------+
| [Footer] Created by Me                                |
+-------------------------------------------------------+
```

2. 🧩 변경된 모든 코드 포함

1) DB: `src/main/resources/db/data.sql`

학습을 위해 충분한 양의 데이터를 준비했습니다.

```sql
-- 게시글 더미 데이터 (총 20개)
INSERT INTO board_tb (title, content, user_id, created_at) VALUES ('첫 번째 게시글', '안녕하세요. ssar의 첫 번째 글입니다.', 1, NOW());
-- ... (중략) ...
```

```
INSERT INTO board_tb (title, content, user_id, created_at) VALUES ('스무 번째 게시글', '게시글 내용 20', 1, NOW());
```

2) DTO: `BoardResponse.java`

Entity를 직접 화면에 노출하지 않고 필요한 데이터만 담는 DTO를 사용합니다.

```
public class BoardResponse {
    @Data
    public static class ListDTO {
        private Integer id;
        private String title;

        // Entity를 DTO로 변환하는 생성자
        public ListDTO(Board board) {
            this.id = board.getId();
            this.title = board.getTitle();
        }
    }
}
```

3) Service: `BoardService.java`

규칙에 따라 한글 메서드명을 사용하며, 모든 게시글을 조회합니다.

```
@Transactional(readOnly = true)
@Service
public class BoardService {
    private final BoardRepository boardRepository;

    public List<BoardResponse.ListDTO> 게시글목록보기() {
        // DB에서 모든 게시글 조회
        var boardList = boardRepository.findAll();
        // Entity 리스트를 DTO 리스트로 변환하여 반환
        return boardList.stream()
                .map(BoardResponse.ListDTO::new)
                .collect(Collectors.toList());
    }
}
```

4) Controller: `BoardController.java`

사용자의 요청을 받아 서비스를 호출하고 뷰에 데이터를 넘깁니다.

```
@Controller
public class BoardController {
    private final BoardService boardService;
```

```
    @GetMapping("/board/list")
    public String list(Model model) {
        // 서비스에서 DTO 목록을 가져옴
        var boardList = boardService.게시글목록보기();
        // Mustache 뷰에서 사용할 이름으로 모델에 추가
        model.addAttribute("boardList", boardList);
        return "board/list";
    }
}
```

5) View: `list.mustache`

Mustache 문법을 사용하여 리스트를 반복 출력합니다.

```
<table class="table table-striped">
    <tbody>
    {{#boardList}} {{! 리스트가 있으면 반복 실행 }}
        <tr>
            <td>{{id}}</td>
            <td>{{title}}</td>
            <td><a href="/board/{{id}}" class="btn btn-primary">상세보기</a></td>
        </tr>
    {{/boardList}}
    </tbody>
</table>
```

3. 🎈 상세비유 쉬운 예시 (Easy Analogy)

"이번 작업은 **도서관에 책 20권을 새로 들여놓고, 그 책들의 제목 목록을 적어서 게시판에 붙인 것**과 같습니다. 아직은 책이 몇 권 안 되어 한 번에 다 보여줄 수 있지만, 나중에 책이 1,000권이 넘어가면 한 번에 다 보여주기 힘들겠죠? 그래서 이번 단계에서는 일단 '모든 목록을 보여주는 게시판'을 먼저 만든 것입니다!"

4. 🔬 기술 딥다이브 (Technical Deep-dive)

- **DTO (Data Transfer Object)**:
  - 계층 간 데이터 교환을 위해 사용하는 객체입니다. Entity는 DB 구조와 밀접하므로 화면 요구사항에 따라 자주 변하는 정보는 DTO에 담아 전달하는 것이 안전합니다.
- **Mustache {{#list}} ... {{/list}}**:
  - 리스트 데이터를 순회하며 내용을 반복해서 그려주는 문법입니다. 데이터가 비어있으면 해당 영역은 화면에 나타나지 않습니다.
- **@Transactional(readOnly = true)**:
  - 데이터를 조회만 할 때 성능을 최적화하고 실수로 데이터를 수정하는 것을 방지하기 위해 사용합니다.